

Clazo (AI-Powered Education Platform)

Clazo is a centralized platform for learning, communication, and career guidance, connecting students, teachers, parents, and administrators. This implementation plan outlines the architecture, database schema, and MVP features for a production-ready web application.

Proposed Unique Features

To make Clazo stand out, I propose adding the following unique features to the MVP:

1. **AI "Knowledge Gap" Mapping**: When students ask the Claude-powered chatbot questions, the AI doesn't just answer; it identifies underlying conceptual gaps and automatically generates a mini "Learning Path" of recommended materials to bridge those gaps.
2. **Socratic AI Tutor**: Instead of giving direct answers, the AI can be toggled into "Socratic Mode", guiding students to find the answer themselves through targeted questioning.

Architecture & Tech Stack

* **Frontend**: Next.js 14+ (App Router), React, Tailwind CSS, Shadcn UI (for clean, modern, accessible components).

* **Backend**: Next.js Route Handlers (Serverless APIs).

* **Database & Auth**: Firebase Authentication (Role-based: Student, Teacher, Parent, Admin), Cloud Firestore (NoSQL Database), Firebase Storage (for materials).

* **AI Integration**: Anthropic Claude API (via Next.js backend for security).

Database Schema (Firestore)

`users` Collection

- `uid` (String, primary key)
- `name` (String)
- `email` (String)
- `role` (Enum: 'student', 'teacher', 'parent', 'admin')
- `createdAt` (Timestamp)

- `profileData` (Map - role specific fields, e.g., `parentId` for students)

`materials` Collection

- `id` (String)
- `title` (String)
- `description` (String)
- `fileUrl` (String)
- `teacherId` (String - ref to `users`)
- `subject` (String)
- `createdAt` (Timestamp)

`assignments` Collection

- `id` (String)
- `title` (String)
- `description` (String)
- `dueDate` (Timestamp)
- `teacherId` (String - ref to `users`)
- `subject` (String)

`submissions` Collection

- `id` (String)
- `assignmentId` (String - ref to `assignments`)
- `studentId` (String - ref to `users`)
- `content` (String / fileUrl)
- `grade` (Number - optional)
- `feedback` (String - optional)
- `submittedAt` (Timestamp)

`ai\_interactions` Collection (For tracking & analytics)

- `id` (String)
- `studentId` (String)

- `query` (String)
- `response` (String)
- `identifiedGaps` (Array of Strings)
- `timestamp` (Timestamp)

Proposed Changes / Execution Steps

The implementation will be divided into the following phases:

Phase 1: Foundation & Authentication

1. Initialize Next.js project with Tailwind CSS.
2. Set up Firebase Auth, Firestore, and Storage.
3. Implement Role-based Authentication Flow (Login, Signup, Role routing).
4. Create base Dashboard Layouts (Sidebar navigation, Header).

Phase 2: Teacher Dashboard & Content Management

1. Build UI for Teachers to upload Study Materials.
2. Build UI for Teachers to create Assignments.
3. Integrate Firebase Storage for file uploads and Firestore for metadata.

Phase 3: Student Dashboard & Assignment System

1. Build Student Dashboard to view materials and assignments.
2. Implement Assignment submission feature.
3. Create Teacher view for reviewing submissions.

Phase 4: Unique AI Chatbot Integration

1. Implement Next.js API route securely connecting to Claude API.
2. Build the AI Chatbot UI in the Student Dashboard.
3. Implement the ****Knowledge Gap Mapping**** prompt logic.

Phase 5: Analytics & Polish

1. Create basic analytics charts for student performance and progress.
2. Refine UI/UX for a premium, modern feel.
3. Final testing and bug fixes.

Verification Plan

Automated/Manual Verification

- I will run the Next.js development server locally.
- Test authentication flows for each role (Student, Teacher).
- Verify file uploads and database writes/reads in the Firebase Emulator or a test project.
- Mock the Claude API response to verify the frontend UI and data parsing before requesting an actual API key.